



LangChain

Prompt Engineering — From Basics to Advanced

With WHY, Code, and Official References

Topics Covered

1. Prompt Datatypes & Message Objects
2. PromptTemplate vs ChatPromptTemplate
3. Why Not Just Use Raw Text?
4. Chaining with LCEL (LangChain Expression Language)
5. Few-Shot Prompting
6. Memory & Conversation History
7. Advanced Prompt Patterns
8. Quick Reference & Cheat Sheet

Before You Start — Orientation for ML Engineers

You already know ML. You have trained models, written pipelines, tuned hyperparameters. LangChain is NOT a new concept — it is a new vocabulary for things you already understand. This section maps what you know to what LangChain calls it.

0.1 Environment Setup — Run This First

Everything in this document is runnable. Install these packages once and follow along:

```
# Core LangChain
```

```
pip install langchain langchain-core langchain-community
```

```
# Model providers (choose one or both)
```

```
pip install langchain-openai      # For GPT-4, GPT-3.5
```

```
pip install langchain-anthropic   # For Claude
```

```
# Optional but useful

pip install langchain-chroma      # Vector store for few-shot selectors

pip install python-dotenv        # Load API keys from .env file


# Set your API key

import os

os.environ['OPENAI_API_KEY'] = 'your-key-here'

# OR create a .env file with: OPENAI_API_KEY=your-key-here
```

 **Official Installation Guide:** https://python.langchain.com/docs/how_to/installation/

0.2 LangChain Package Ecosystem — The Map

The #1 confusion for new LangChain users: why are there so many packages? Here is what each one does:

Package	What It Contains
langchain-core	Base classes: Runnable, BaseMessage, PromptTemplate. The skeleton. Always import from here first.
langchain	High-level abstractions: chains, agents, few-shot helpers. The muscles on top of the skeleton.
langchain-community	Third-party integrations: vector stores, document loaders, external memory. Community-maintained.
langchain-openai	OpenAI-specific wrappers: ChatOpenAI, OpenAIEmbeddings. Official integration.
langchain-anthropic	Anthropic Claude wrappers: ChatAnthropic. Official integration.

 **ML Intuition:** *Think of langchain-core like numpy — foundational, stable, everything else builds on it. langchain-openai is like a hardware-specific driver — it adapts the core API to one provider.*

 **Package Architecture Docs:** <https://python.langchain.com/docs/concepts/architecture/>

0.3 What is a Runnable? — The Most Important LangChain Concept

Every LangChain component (prompts, models, parsers, retrievers) inherits from one base class: Runnable. This is what makes everything chainable.

```
# Every LangChain component has these built-in automatically

component.invoke(input)          # Run once, return output

component.stream(input)          # Stream output token by token

component.batch([input1, input2]) # Run multiple inputs in parallel


# AND Runnables can be piped with |

chain = component_a | component_b | component_c
```

 **ML Intuition:** *If you have used sklearn Pipeline, you already understand Runnables. sklearn: Pipeline([('scaler', StandardScaler()), ('model', LogisticRegression())]). LangChain: prompt | model | parser. Same idea — compose steps, data flows through automatically.*

sklearn Pipeline	LangChain LCEL
Pipeline([('step', Transformer())])	prompt model parser
Each step: fit() + transform()	Each step: invoke()
X flows through all steps	dict flows through all steps
Final output: prediction	Final output: string / object
Steps must be sklearn-compatible	Steps must be Runnable

0.4 LangChain Terms Translated for ML Engineers

First time you see these terms in code, refer back here:

Runnable -- Any LangChain component that can be invoked, streamed, or batched. Base class of everything.

LCEL -- LangChain Expression Language — the | pipe syntax for chaining Runnables. Like Unix pipes for AI.

Chain -- A sequence of Runnables connected with |. The LangChain equivalent of an sklearn Pipeline.

PromptTemplate -- A Runnable that takes a dict of variables and outputs a formatted string.

ChatPromptTemplate -- A Runnable that takes a dict of variables and outputs a list of typed messages.

BaseMessage -- Base class for all message types. Like a typed wrapper around a string with a role attached.

ChatModel -- Any LLM that accepts a list of BaseMessage objects. e.g. ChatOpenAI, ChatAnthropic.

OutputParser -- A Runnable that converts a model response to a usable Python type (str, JSON, Pydantic).

MessagesPlaceholder -- A slot inside ChatPromptTemplate filled with past messages at runtime — enables memory.

RunnableWithMessageHistory -- Wrapper that auto-loads and saves conversation history per session around any chain.

0.5 The Big Picture — How Everything Connects

Before reading any code, burn this flow into your head. Every LangChain app follows this pattern:

```
YOUR INPUT (dict with variables)
    |
    v
ChatPromptTemplate      <-- fills variables into message templates
    |
```

```

      v

[SystemMessage, HumanMessage, ...]  <-- typed list of messages

      |

      v

ChatModel (GPT-4 / Claude)  <-- sends messages to the LLM API

      |

      v

AIMessage(content=...)        <-- model response, typed object

      |

      v

OutputParser                  <-- converts to str / JSON / Pydantic

      |

      v

YOUR OUTPUT

```

In code, this entire flow is one line:

```

chain = chat_prompt_template | chat_model | output_parser

result = chain.invoke({'variable': 'value'})

```

 **ML Intuition:** *The | pipe builds a lazy computation graph — just like defining a PyTorch model does not run a forward pass. Execution only happens when you call .invoke(), .stream(), or .batch().*

Part 1 — Prompt Datatypes in LangChain

LangChain has a message type system at its foundation. Every prompt, response, and chat history is built from these objects. Understanding them is non-negotiable.

 **WHY:** *Chat models like GPT-4 and Claude don't receive plain text — they receive a structured list of messages with roles. LangChain models this with typed objects so you*

never lose track of who's speaking. Without types, you'd be manually formatting JSON every time.

 **Official Docs — Messages:** <https://python.langchain.com/docs/concepts/messages/>

1.1 HumanMessage

Represents what the user says in the conversation.

```
from langchain_core.messages import HumanMessage

msg = HumanMessage(content='What is a transformer?')

print(msg.content)      # 'What is a transformer?'

print(type(msg))        # <class 'HumanMessage'>
```

 **WHY:** *The model needs to distinguish the user's voice from the system's. Wrapping in HumanMessage explicitly labels this as user input.*

 **Classroom Analogy:** *HumanMessage = the question a student raises in class.*

1.2 AIMessage

Represents the model's response. When you call `model.invoke()`, LangChain automatically wraps the result in this type.

```
from langchain_core.messages import AIMessage

msg = AIMessage(content='A transformer is a deep learning architecture...')

# This is exactly what model.invoke() returns - wrapped automatically
```

 **WHY:** *Keeping AI responses typed lets you inspect them, log them, pass them back into history, or chain them into the next prompt — without converting back and forth from strings.*

1.3 SystemMessage

Sets the persona, behavior, or constraints for the model. It's the 'backstage instruction' given before the conversation starts.

```
from langchain_core.messages import SystemMessage
```

```
msg = SystemMessage(content='You are a concise AI tutor who teaches with examples.')
```

💡 **WHY:** System messages carry higher authority than human messages. The model treats them as hard rules. Putting instructions inside HumanMessage is weaker and less reliable.

🎓 **Classroom Analogy:** SystemMessage = the job description you hand a new employee on Day 1.

1.4 ChatPromptValue

When you call `.format_messages()` on a `ChatPromptTemplate`, it produces a `ChatPromptValue` — essentially a typed list of the above message objects that gets forwarded to the model.

```
# Internally, this is what ChatPromptTemplate produces:
```

```
messages = [  
    SystemMessage(content='You are an AI tutor.'),  
    HumanMessage(content='Explain LSTMs.')  
]
```

```
# This list is exactly what a ChatModel expects as input
```

Type	Represents
HumanMessage	User's input / question
AIMessage	Model's response (returned automatically)
SystemMessage	Behavior instructions / persona
ChatPromptValue	Formatted list of messages (output of ChatPromptTemplate)

Part 2 — PromptTemplate vs ChatPromptTemplate

Both are templates with variables — but they produce completely different output types for completely different model types.

 **Official Docs — Prompt Templates:** https://python.langchain.com/docs/concepts/prompt_templates/

2.1 PromptTemplate — Plain String Output

Designed for completion-style LLMs. Takes variables, fills them in, and returns a single Python string.

```
from langchain.prompts import PromptTemplate

pt = PromptTemplate(
    input_variables=['name', 'topic'],
    template='Hi {name}! Explain {topic} like I am 10 years old.'
)

result = pt.format(name='Rahul', topic='attention mechanism')

print(result)    # 'Hi Rahul! Explain attention mechanism like I am 10.'

print(type(result))  # <class 'str'>
```

 **WHY:** When you have a simple one-turn prompt with no role separation, `PromptTemplate` is sufficient. It's also the base for building reusable, chainable prompt components.

2.2 ChatPromptTemplate — List of Messages Output

Designed for chat-based LLMs. Produces a structured list of typed message objects — not a plain string.

```
from langchain.prompts import ChatPromptTemplate

cpt = ChatPromptTemplate.from_messages([
    ('system', 'You are an expert in {domain}.'),
    ('human', 'Explain {topic} with a real-world example.')
```


1)

```
messages = cpt.format_messages(domain='deep learning', topic='attention')
```

```
for m in messages:
```

```
    print(type(m).__name__, ': ', m.content)
```

```
# SystemMessage : You are an expert in deep learning.
```

```
# HumanMessage : Explain attention with a real-world example.
```

💡 **WHY:** Chat models expect a list of role-labeled messages — not a single block of text. *ChatPromptTemplate* is the proper way to build this list with dynamic variables.

🎓 **Classroom Analogy:** *PromptTemplate* = writing a letter (one block).
ChatPromptTemplate = scripting a conversation (multiple speakers).

2.3 Side-by-Side Comparison

Feature	PromptTemplate vs ChatPromptTemplate
Output type	str vs List[BaseMessage]
Used with	LLM (completion) vs ChatModel (GPT-4, Claude)
Role support	No roles vs system / human / ai
Real-world use	Rare today vs Standard for modern apps
Chainable (LCEL)	Yes vs Yes

Part 3 — Why Not Just Pass Raw Text?

This is the most important 'why' question. Raw strings work for one-off experiments. Templates are for everything else. Here are the 5 reasons that matter.

Problem 1 — Variables Break Without a System

With raw strings you manually concatenate or f-string every variable. Inside a LangChain chain, a plain f-string is not a component — you cannot pipe it.

```
# Raw string approach — cannot be piped
```

```

topic = 'backpropagation'

prompt = f'Explain {topic} simply.' # just a str, not Runnable

# This FAILS — you cannot | a plain string

chain = f'Explain {topic}...' | model | parser # ❌ TypeError

# PromptTemplate IS a Runnable — pipeable

chain = prompt_template | model | parser # ✅ Works perfectly

```

💡 **WHY:** *PromptTemplate inherits from Runnable. That means it has .invoke(), .stream(), .batch() built-in — and it works seamlessly with the | pipe operator in LCEL.*

Problem 2 — Chat Models Need Role Separation

When you pass a raw string to ChatOpenAI, LangChain secretly wraps it in HumanMessage. The model has no idea which part is instruction vs input.

```

# You CANNOT split roles with a raw string

raw = 'You are an expert tutor. Explain gradient descent.'

# The model reads this as one HumanMessage — your system

# instruction has no authority!

# With ChatPromptTemplate, roles are explicit

# SystemMessage gets higher authority than HumanMessage

```

💡 **WHY:** *System messages are treated differently by the model internally. Blending them into one string loses that distinction and produces weaker, less predictable outputs.*

Problem 3 — Zero Reusability

```

# Raw — rewrite every single time

p1 = f'Explain {topic} to a beginner'

p2 = f'Explain {topic} to an expert'

```

```
# Template — define once, reuse forever

template = PromptTemplate.from_template('Explain {topic} to a {audience}')

template.invoke({'topic': 'LSTMs', 'audience': 'beginner'})

template.invoke({'topic': 'LSTMs', 'audience': 'expert'})
```

Problem 4 — No Validation

```
# Raw string — silent bug, no error

prompt = f'Explain {topic} simply.' # name silently dropped — no error!


# PromptTemplate — loud error

pt = PromptTemplate(input_variables=['topic', 'name'], template='Hi {name}! Explain {topic}.')

pt.format(topic='attention') # ❌ KeyError: 'name' — caught immediately
```

💡 **WHY:** *In production, silent bugs are catastrophic. Templates enforce that all variables are provided before execution.*

Problem 5 — Not Serializable

Raw strings cannot be saved, versioned, or loaded from disk. PromptTemplates are first-class objects with full serialization support.

```
pt.save('my_prompt.json') # Save to disk

loaded = load_prompt('my_prompt.json') # Load it back

# Raw f-strings: impossible to do this cleanly
```

📎 **Serialization Docs:** https://python.langchain.com/docs/how_to/prompts_serialization/

Part 4 — Chaining with LCEL

LCEL (LangChain Expression Language) is LangChain's modern composition system. It uses the `|` pipe operator to connect components — just like Unix pipes.

 **WHY:** Without LCEL you'd write verbose chains of function calls, manual error handling, and custom streaming code. LCEL gives you lazy evaluation, automatic streaming, batching, and async support for free.

4.1 Basic Chain

```
from langchain.prompts import ChatPromptTemplate

from langchain_openai import ChatOpenAI

from langchain_core.output_parsers import StrOutputParser


prompt = ChatPromptTemplate.from_messages([

    ('system', 'You are a helpful AI tutor.'),

    ('human', 'Explain {topic} in 3 bullet points.')

])


model = ChatOpenAI(model='gpt-4o')

parser = StrOutputParser()


# The chain - each | passes output to next component

chain = prompt | model | parser


result = chain.invoke({'topic': 'gradient descent'})

print(result) # Plain string output
```

4.2 What Flows Through the Pipe

Step	Input → Output
invoke({'topic': '...'})	dict → ChatPromptValue
prompt	ChatPromptValue → List[BaseMessage]

model	List[BaseMessage] → AIMessage
parser	AIMessage → str (plain string)

 **Classroom Analogy:** LCEL is like an assembly line. Each station does one job and passes the result forward. The | pipe is the conveyor belt.

4.3 Streaming

```
# LCEL chains support streaming natively — no extra code needed

for chunk in chain.stream({'topic': 'backpropagation'}):

    print(chunk, end='', flush=True)
```

4.4 Parallel Chains with RunnableParallel

```
from langchain_core.runnables import RunnableParallel

parallel_chain = RunnableParallel({

    'simple':    simple_prompt    | model | parser,

    'detailed': detailed_prompt | model | parser

})

result = parallel_chain.invoke({'topic': 'backpropagation'})

print(result['simple'])    # Simple explanation

print(result['detailed']) # Detailed explanation
```

 LCEL Primitives: <https://python.langchain.com/docs/concepts/runnables/>

Part 5 — Few-Shot Prompting

Few-shot prompting = giving the model examples inside the prompt so it learns the expected pattern before answering. It's the single most powerful prompting technique.

💡 **WHY:** LLMs are pattern-completion engines. Showing them 2-3 examples of the expected format is far more reliable than explaining the format in words alone.

📎 Official Docs — Few-Shot: https://python.langchain.com/docs/how_to/few_shot_examples_chat/

5.1 FewShotChatMessagePromptTemplate

```
from langchain.prompts import FewShotChatMessagePromptTemplate, ChatPromptTemplate
```

```
examples = [

    {'input': 'happy', 'output': 'sad'},

    {'input': 'tall', 'output': 'short'},

]

# Template for each example pair

example_prompt = ChatPromptTemplate.from_messages([

    ('human', '{input}'),

    ('ai', '{output}'),

])
```

```
few_shot = FewShotChatMessagePromptTemplate(

    example_prompt=example_prompt,

    examples=examples

)
```

```
# Combine with a final question

final_prompt = ChatPromptTemplate.from_messages([

    ('system', 'Give the opposite of each word.'),
```

```

        few_shot,

        ('human', '{word}'))

    ])

chain = final_prompt | model | parser

print(chain.invoke({'word': 'fast'})) # -> 'slow'

```

5.2 Dynamic Few-Shot with Example Selector

For large example sets, use a `SemanticSimilarityExampleSelector` to automatically pick the most relevant examples for each query.

```

from langchain.prompts import SemanticSimilarityExampleSelector

from langchain_openai import OpenAIEmbeddings

from langchain_community.vectorstores import Chroma

selector = SemanticSimilarityExampleSelector.from_examples(

    examples,

    OpenAIEmbeddings(),

    Chroma,

    k=2 # Pick top 2 most similar examples

)

few_shot = FewShotChatMessagePromptTemplate(

    example_selector=selector, # Dynamic!

    example_prompt=example_prompt

)

```

 **Example Selectors:** https://python.langchain.com/docs/how_to/example_selectors/

Part 6 — Memory & Conversation History

By default, LLMs are stateless — every call is independent. Memory lets your chain remember previous turns, enabling real multi-turn conversations.

 **WHY:** *Without memory, a user saying 'continue from where we left off' would return a blank response. Memory makes chatbots feel intelligent rather than amnesiac.*

 **Official Docs — Memory / Chat History:** https://python.langchain.com/docs/concepts/chat_history/

6.1 MessagesPlaceholder

This is the key ingredient — a reserved slot in your ChatPromptTemplate that gets filled with the entire conversation history at runtime.

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([

    ('system', 'You are a helpful AI tutor.'),

    MessagesPlaceholder(variable_name='history'), # ← slot for past messages

    ('human', '{input}'))

1)
```

 **Classroom Analogy:** *MessagesPlaceholder is like a bookmarked blank page in your script. Before each turn, the full chat history gets inserted there.*

6.2 RunnableWithMessageHistory

Wrap any chain with this to automatically load and save conversation history per session.

```
from langchain_community.chat_message_histories import ChatMessageHistory

from langchain_core.runnables.history import RunnableWithMessageHistory

from langchain_openai import ChatOpenAI

from langchain_core.output_parsers import StrOutputParser
```



```

model = ChatOpenAI()

parser = StrOutputParser()

chain = prompt | model | parser


store = {}

def get_session_history(session_id):

    if session_id not in store:

        store[session_id] = ChatMessageHistory()

    return store[session_id]


chain_with_memory = RunnableWithMessageHistory(

    chain,

    get_session_history,

    input_messages_key='input',

    history_messages_key='history'

)


# Turn 1

chain_with_memory.invoke(

    {'input': 'My name is Rahul'},

    config={'configurable': {'session_id': 's1'}}

)


# Turn 2 – model remembers!

chain_with_memory.invoke(

    {'input': 'What is my name?'},

```

```
        config={'configurable': {'session_id': 's1'}}
    )

# Output: 'Your name is Rahul.'
```

 **Conversation History How-To:** https://python.langchain.com/docs/how_to/message_history/

Part 7 — Advanced Prompt Patterns

7.1 Partial Prompts

Pre-fill some variables of a template, leaving others to be filled later. Useful for shared system prompts across many chains.

```
from langchain.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([

    ('system', 'You are a {role} assistant.'),

    ('human', '{question}'))

])

# Pre-fill 'role' — reuse this partial for all tutor chains

tutor_prompt = prompt.partial(role='helpful AI tutor')

# Only need to supply 'question' now

chain = tutor_prompt | model | parser

chain.invoke({'question': 'Explain RNNs'})
```

 **WHY:** *Partial prompts let you bake in constants once and share the template without duplication. Perfect for multi-tenant or multi-persona apps.*

 **Partial Prompt Variables:** https://python.langchain.com/docs/how_to/prompts_partial/

7.2 Output Parsers

Output parsers extract and structure the model's raw text response into typed Python objects.

```
from langchain_core.output_parsers import JsonOutputParser

from langchain_core.pydantic_v1 import BaseModel, Field


class Explanation(BaseModel):

    concept: str = Field(description='The concept name')

    summary: str = Field(description='One sentence summary')

    example: str = Field(description='A real-world example')


parser = JsonOutputParser(pydantic_object=Explanation)


prompt = ChatPromptTemplate.from_messages([

    ('system', 'Answer in JSON. {format_instructions}'),

    ('human', 'Explain {topic}'))

]).partial(format_instructions=parser.get_format_instructions())


chain = prompt | model | parser

result = chain.invoke({'topic': 'gradient descent'})

print(result.concept) # 'gradient descent'

print(result.summary) # structured output!
```

 **WHY:** *Raw text outputs are fragile for downstream processing. Parsers give you typed, validated Python objects that play nicely with databases, APIs, and other chains.*

 **Output Parsers:** https://python.langchain.com/docs/concepts/output_parsers/

7.3 Chain-of-Thought with Step-Back Prompting

Guide the model to reason step-by-step before giving a final answer — dramatically improves accuracy on complex tasks.

```
cot_prompt = ChatPromptTemplate.from_messages([
    ('system', 'Think step by step before answering. Show your reasoning.'),
    ('human', '{question}')
])
```

```
# The model will now reason aloud before concluding
```

```
chain = cot_prompt | model | parser
```

 **Prompt Engineering Guide (Anthropic):** <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>

7.4 LangChain Hub — Reuse Community Prompts

LangChain Hub is a registry of production-tested prompts. Pull and reuse them directly in your chains.

```
from langchain import hub
```

```
# Pull a community-maintained RAG prompt
```

```
rag_prompt = hub.pull('rlm/rag-prompt')
```

```
# Use it directly in your chain
```

```
chain = rag_prompt | model | parser
```

 **LangChain Hub:** <https://smith.langchain.com/hub>

Part 8 — Quick Reference Cheat Sheet

Concept	One-liner for students
HumanMessage	What the user says

SystemMessage	Instructions / persona given to the model
AIMessage	What the model replies (auto-wrapped)
PromptTemplate	A plain string with fillable blanks → outputs str
ChatPromptTemplate	A conversation script with blanks → outputs List[Message]
LCEL	Assembly line: each component does one job and passes forward
StrOutputParser	Strips AIMessage wrapper → gives you plain string
MessagesPlaceholder	Reserved slot in prompt that gets filled with chat history
Few-Shot	Teaching the model by example inside the prompt
Partial Prompt	Pre-fill some variables, leave others for later
JsonOutputParser	Forces model to respond in JSON → typed Python object

Official Documentation Index

 **LangChain Concepts Home:** <https://python.langchain.com/docs/concepts/>

 **Messages & Datatypes:** <https://python.langchain.com/docs/concepts/messages/>

 **Prompt Templates:** https://python.langchain.com/docs/concepts/prompt_templates/

 **LCEL Introduction:** <https://python.langchain.com/docs/concepts/lcel/>

 **Runnables & Primitives:** <https://python.langchain.com/docs/concepts/runnables/>

 **Few-Shot Prompting:** https://python.langchain.com/docs/how_to/few_shot_examples_chat/

 **Chat History / Memory:** https://python.langchain.com/docs/how_to/message_history/

 **Output Parsers:** https://python.langchain.com/docs/concepts/output_parsers/

 **Partial Prompt Variables:** https://python.langchain.com/docs/how_to/prompts_partial/

 **Prompt Serialization:** https://python.langchain.com/docs/how_to/prompts_serialization/

 **Example Selectors:** https://python.langchain.com/docs/how_to/example_selectors/

 **LangChain Hub:** <https://smith.langchain.com/hub>

Built for Rahul | RByteai.com
